Partial Assembly by L^2-Projection for Inexact-Newton Solvers in Elasticity

Patrick Zulian*

September 7, 2026

Abstract

Matrix-free Newton–Krylov solvers for finite-strain elasticity spend most of their time applying the tangent operator, and each application carries a quadrature loop because the material tangent varies within every element. We replace that tangent by its element-wise L^2 projection onto the constants. The projection separates the integrand into a small per-element tensor, assembled once per Newton step, and a purely geometric tensor that belongs to the reference element and can therefore be integrated exactly and symbolically at code-generation time. The quadrature loop leaves the kernel entirely, and the resulting operator application reads only the stored tensor and the increment: no geometry, no state, no material parameters.

The reference tensor is far smaller than its size suggests. Read as a matrix on the (node, direction) pair index it is the Gram matrix of the element's gradient functions, so its rank equals the dimension of the span of those functions: one for linear simplices, four for quadratic tetrahedra, seven for trilinear hexahedra. We give an exact rational factorisation of that rank structure and show that the projection is an identity, not an approximation, on affine simplices — for nonlinear materials as much as linear ones — which supplies an exact reference against which the construction can be verified. We also show that the symmetry that halves the storage is a property of the *formulation* rather than of the method, and that assuming it for a residual-based material such as Kelvin–Voigt viscoelasticity yields a different operator, with 5.3×10^{-2} relative error on an element where the projection is provably exact.

A code generator implements the construction for five element types, choosing between two contraction orderings per element by symbolic operation count. On a 10.4-million-degree-of-freedom neo-Hookean problem the generated kernels reach 219.6 MDOF/s on quadratic tetrahedra against 137.4 for a hand-optimised partial-assembly implementation of the same material, with the assembly phase also faster (0.392 s against 0.653 s), and parity on trilinear hexahedra. Parallel efficiency is 96% on 72 Arm Neoverse-V2 cores. The approximation converges to the exact operator at $O(h^{1.07})$ under refinement.

Contents

*Affiliation to be completed.

1 Introduction

A Newton–Krylov solve of a finite-strain elasticity problem has a characteristic cost profile. Each Newton step forms a residual once and then applies the tangent operator tens to hundreds of times inside a Krylov method. In a matrix-free implementation each of those applications re-evaluates the material tangent at every quadrature point of every element, because the tangent depends on the deformation gradient and the deformation gradient varies within the element. The work is repeated identically across all Krylov iterations of a step: the state does not change, only the increment does.

Sparse assembly is the classical way to pay that cost once, but for high-order or vector-valued problems the assembled matrix is expensive in memory and its application is bandwidth-bound. *Partial assembly* — storing a compact per-element quantity that captures the geometry and the state, and applying it with a small tensor contraction — occupies the middle ground, and is standard practice in high-order matrix-free codes for linear operators [21, 10, 17, 16, 7]. Extending it to a nonlinear material is not immediate, because the object that would be stored is not constant over the element.

This paper makes that extension by projection. We replace the pulled-back material tangent $\overline{S}(\boldsymbol{\xi})$ by its element average $\overline{S}^0$, which is its L^2 projection onto the constants on the element. Because $\overline{S}^0$ leaves the integral, the remaining integrand involves only reference basis-function gradients: a tensor $\overline{W}$ that depends on the element type and on nothing else, and can therefore be integrated exactly in rational arithmetic before the program is compiled. What is stored per

element is $\overline{S}^0$ alone — 45 numbers in three dimensions for a symmetric material — and what the operator application reads is that store and the increment.

The resulting method is inexact by construction, which is why the target is an inexact-Newton solver [9, 11, 14]: a framework that already tolerates an approximate Jacobian action provided the approximation is controlled. We show the approximation is first-order in the mesh size, and we measure it.

Contributions

1. A partial-assembly scheme for nonlinear elasticity obtained by L^2 projection of the pulled-back tangent onto element-wise constants (Section 4), with a proof that the projection is exact — not merely accurate — on affine simplices for arbitrary hyperelastic materials (Proposition 1).
2. Identification of the reference tensor as a Gram matrix, giving its rank in closed form as the dimension of the span of the element's gradient functions (Proposition 4), together with an exact rational factorisation that turns the operator application into four staged contractions.
3. A symmetry analysis (Proposition 3) showing that the storage-halving symmetry of $\overline{S}^0$ holds for energy-based materials and fails for residual-based ones, with a measured counterexample.
4. Two contraction orderings with a per-element selection rule based on symbolic operation count after common-subexpression elimination (Section 6.2), and evidence that the count must be taken after elimination rather than before.
5. An implementation in a finite-element code generator covering TRI3, QUAD4, TET4, TET10 and HEX8, and a measurement campaign (Section 8) comparing against hand-optimised kernels for the same materials at 10.4 million degrees of freedom, on two architectures, with a stated experimental protocol.

2 Related work

Matrix-free operator application. The idea of storing per-quadrature-point geometric data and applying an operator by tensor contraction goes back to spectral-element methods [21, 10] and is the basis of modern high-order matrix-free frameworks: `deal.II`'s cell-based interface [17, 18], MFEM [2], and libCEED [7], whose "operator data" or "Q-data" plays the role of our $\overline{S}$. The efficiency of these methods rests on sum factorisation, whose asymptotic advantage appears at polynomial degrees well above those considered here [19, 16]. Our setting is complementary: low to moderate order, where the element tensors are small enough to be handled exactly and symbolically, and where the dominant cost is the repeated evaluation of a nonlinear constitutive law rather than the basis-function contraction.

Nonlinear matrix-free solvers. Brown [6] analyses matrix-free Newton–Krylov methods for nodal high-order elements in three dimensions and identifies the tangent evaluation as the bottleneck. Jacobian-free Newton–Krylov methods [14] avoid forming the tangent at all, at the price of a directional-difference approximation whose accuracy is delicate; our scheme instead forms an approximate tangent explicitly, which is cheaper to apply repeatedly and whose error is a property of the discretisation rather than of a difference parameter.

Inexact Newton. The convergence theory of Newton methods with inexactly solved — or inexactly formed — linear systems is classical [9], with the choice of forcing terms treated in

[11]. Our operator perturbation is $O(h)$ (Proposition 2), which places it within the regime these results address.

Code generation for finite elements. Automatic generation of element kernels from a symbolic weak form is established practice: the FEniCS form compiler [13], the Unified Form Language [3], and Firedrake [22]. What distinguishes the present work is that the generator performs the projection, the exact rational integration, the rank factorisation and the contraction-ordering choice symbolically, and emits element-specific code in which the reference tensor appears only as folded literal constants.

Low-rank structure and reduced precision. The factorisation of Section 5 is an exact rank-revealing decomposition of a Gram matrix rather than a numerical low-rank approximation [15]; the rank is small for structural reasons, not because the tensor happens to be compressible. Storing the assembled tensor at reduced precision connects to the broader mixed-precision literature [12, 1]; our finding (Section 6.5) is that `fp32` is the useful operating point and `fp16` is not, for reasons of bandwidth saturation rather than accuracy.

3 Problem setting

Let $\Omega \subset \mathbb{R}^d$, $d \in \{2, 3\}$, be discretised by a mesh of elements K, each the image of a reference cell $\widehat{K}$ under a map $\boldsymbol{x}(\boldsymbol{\xi})$ with Jacobian $J = \partial \boldsymbol{x}/\partial \boldsymbol{\xi}$ and $\mathrm{d}V = \det J \, \mathrm{d}\boldsymbol{\xi}$. Let $\{\varphi_p\}_{p=1}^{N}$ be the scalar reference basis, with the displacement field discretised component-wise on the same space.

For a hyperelastic material with strain energy $\Psi(F)$ and first Piola–Kirchhoff stress $P = \partial\Psi/\partial F$ [20, 4], the matrix-free action of the tangent operator on an increment h is

$$(\mathcal{H}h)_{i,p} = \int_K S_{ijkl}\big(F(\boldsymbol{x})\big)\, \frac{\partial h_k}{\partial x_l}\, \frac{\partial \varphi_p}{\partial x_j}\, \mathrm{d}V, \qquad S_{ijkl} = \frac{\partial P_{ij}}{\partial F_{kl}}, \tag{1}$$

where i indexes the output component and p the test function.

Pulling the derivatives back to the reference cell with $\partial/\partial x_j = \sum_n J^{-1}_{nj}\, \partial/\partial \xi_n$ and collecting all geometric factors into the tangent gives

$$\overline{S}_{ikmn} = \sum_{j,l} S_{ijkl}\, J^{-1}_{nj}\, J^{-1}_{ml}\, \det J, \tag{2}$$

so that (1) becomes

$$(\mathcal{H}h)_{i,p} = \int_{\widehat{K}} \overline{S}_{ikmn}(\boldsymbol{\xi})\, \frac{\partial h_k}{\partial \xi_m}\, \frac{\partial \varphi_p}{\partial \xi_n}\, \mathrm{d}\boldsymbol{\xi}. \tag{3}$$

The tensor $\overline{S}$ is the material tangent with the geometry already folded in; it is the analogue of the "operator data" of [7] for the second variation. Every factor under the integral in (3) except $\overline{S}$ is a property of the reference element alone. What forces a quadrature loop is the dependence of $\overline{S}$ on $\boldsymbol{\xi}$, which enters through F.

3.1 Cost structure

Within one Newton step the state, and hence $\overline{S}$, is fixed while (3) is applied once per Krylov iteration. Writing n_{it} for the number of applications per step, a matrix-free implementation performs n_{it} evaluations of the constitutive tangent per quadrature point, all of them identical. This is the redundancy the present method removes: it performs one, stores its projection, and makes the remaining $n_{\mathrm{it}} - 1$ applications independent of the state entirely.

4 Partial assembly by L^2 projection

Let Π^0_K denote the $L^2(\widehat{K})$ projection onto the constants, that is the element average, and set

$$\overline{S}^0 \;=\; \Pi^0_K \overline{S} \;=\; |\widehat{K}|^{-1} \int_{\widehat{K}} \overline{S}\, \mathrm{d}\boldsymbol{\xi}. \tag{4}$$

Replacing $\overline{S}$ by $\overline{S}^0$ in (3) takes it out of the integral and separates the sum:

$$(\mathcal{H}h)_{i,p} \;\approx\; \sum_{k,m,n} \overline{S}^0_{ikmn} \sum_j h_{k,j}\, \overline{W}_{jmpn}, \qquad \overline{W}_{jmpn} = \int_{\widehat{K}} \frac{\partial \varphi_j}{\partial \xi_m} \frac{\partial \varphi_p}{\partial \xi_n}\, \mathrm{d}\boldsymbol{\xi}. \tag{5}$$

Two objects appear, with entirely different lifetimes. $\overline{S}^0$ is one small tensor per element, computed once per Newton step from the current state. $\overline{W}$ depends only on the element type; it is integrated exactly, in rational arithmetic, at code-generation time. The quadrature loop is therefore absent from the operator application, and so are the geometry, the state and the material parameters: the application reads $\overline{S}^0$ and h and nothing else.

4.1 Exactness on affine simplices

Proposition 1 (Exactness)**.** *Let K be an affinely mapped simplex discretised with the lowest-order (P1) basis, and let the material be any function of the deformation gradient. Then* (5) *holds with equality.*

Proof. For an affine map J is constant on K, and for a P1 displacement field the gradients $\partial h_k/\partial \xi_m$ are constant, so F is constant on K. Hence $S(F)$ is constant, and by (2) so is $\overline{S}$. The L^2 projection of a constant onto the constants is that constant, so $\overline{S}^0 = \overline{S}$ pointwise and (5) reproduces (3). □

The proposition is a statement about the geometry, not about the constitutive law: it holds for a nonlinear material exactly as for a linear one. Its practical importance is methodological. It supplies a family of configurations in which the approximate operator must agree with the exact one to round-off, independently of any other component of the implementation, and it is the control against which every measurement in Section 8 is taken.

4.2 Consistency on curved and higher-order elements

Proposition 2 (First-order consistency)**.** *Let $\mathcal{T}_h$ be a shape-regular family of meshes and suppose $\overline{S} \in W^{1,\infty}(K)$ uniformly. Then the operator perturbation induced by the projection satisfies*

$$\big\|(\mathcal{H} - \mathcal{H}^0)h\big\|_K \;\le\; C\, h_K\, |\overline{S}|_{W^{1,\infty}(K)}\, \|\nabla h\|_K\, \|\nabla \varphi\|_K, \tag{6}$$

with C independent of h_K and of the material.

Proof sketch. Π^0_K reproduces constants, so by the Bramble–Hilbert lemma [5, 8] $\|\overline{S} - \Pi^0_K \overline{S}\|_{L^\infty(K)} \le C h_K |\overline{S}|_{W^{1,\infty}(K)}$. Substituting into the difference of (3) and (5) and bounding the remaining integral by Cauchy–Schwarz gives the claim. □

The rate is confirmed experimentally in Section 8.4, where the measured order is 1.07. Within an inexact-Newton framework [9] an $O(h)$ perturbation of the Jacobian action is compatible with retaining a linear rate of convergence, with the constant improving under refinement; establishing the resulting forcing-term theory for this specific perturbation is outside the scope of this paper and is discussed in Section 10.

4.3 Symmetry of the stored tensor

Proposition 3 (Symmetry)**.** *The element matrix implied by* (5) *is symmetric under exchange of the two (component, node) pairs if and only if*

$$\overline{S}^0_{ikmn} = \overline{S}^0_{kinm}. \tag{7}$$

Proof. The entry is $\overline{S}^0_{ikmn}\overline{W}_{jmpn}$, and $\overline{W}_{jmpn} = \overline{W}_{pnjm}$ by (5). Exchanging $(i,j) \leftrightarrow (k,p)$ and relabelling gives the stated condition. □

The index pairing deserves emphasis because the plausible alternative is wrong. The pair that exchanges is (i,n) against (k,m): the output component with the *test* direction, and the increment component with the *increment* direction. Packing on (i,k) against (m,n) instead is a consistent error — it is invisible to any test that compares one form of the construction against another, since both share it — and is detected only by comparison against (1) integrated independently. Under (7), $\overline{S}^0$ has 45 independent components in three dimensions and 10 in two, against 81 and 16 unsymmetric.

Whether (7) holds is a property of the *formulation*, not of the method. A material specified by a strain energy has a flux that is a gradient, so its tangent is a Hessian and (7) follows from equality of mixed partials. A material specified directly by a residual has no such guarantee: its flux is not a gradient, its tangent is a Jacobian, and it need not be symmetric. Kelvin–Voigt viscoelasticity is a case in point. For the parameters used in Section 8 we measure

$$\max_{ijkl} \left| A_{ijkl} - A_{klij} \right| = 0.25, \tag{8}$$

so folding the tangent into 45 components averages its antisymmetric part away and yields a *different operator*, with 5.3×10^{-2} relative error on TET4 — an element on which Proposition 1 guarantees the projection itself is exact.

The generator therefore determines symmetry per material, symbolically, and stores 45 or 81 components accordingly, defaulting to unsymmetric. The two possible mistakes are not comparable in consequence: storing a full square for a symmetric tangent wastes memory, whereas storing half of an unsymmetric one is silently wrong.

5 Structure of the reference tensor

5.1 Sparsity

Because $\overline{W}$ is integrated exactly its entries are rationals, and there are remarkably few distinct ones (Table 1). On simplices the majority vanish.

element	entries	distinct values	zeros	projection
TRI3	36	3	20	exact
TET4	144	3	108	exact
QUAD4	64	6	0	inexact
HEX8	576	10	0	inexact
TET10	900	9	519	inexact

Table 1: The symbolically integrated reference tensor $\overline{W}$. At most ten distinct rational values occur for any element considered.

5.2 Rank

The sparsity is a symptom of a stronger property.

Proposition 4 (Rank)**.** *Read as a matrix on the pair index (j,m), $\overline{W}$ is the Gram matrix in $L^2(\widehat{K})$ of the family of gradient functions $\{\partial\varphi_j/\partial\xi_m\}$. Consequently*

$$\operatorname{rank}\overline{W} \;=\; \dim\operatorname{span}\Big\{\tfrac{\partial\varphi_j}{\partial\xi_m}\Big\}. \tag{9}$$

Proof. Immediate from the definition in (5), which is precisely the L^2 inner product of the (j,m)th and (p,n)th members of the family, and from the standard fact that a Gram matrix has the rank of the dimension of the span of the vectors it is built from. □

element	order Nd	rank r	span of the gradient functions
TRI3	6	1	constants
TET4	12	1	constants
QUAD4	8	3	$1,\ \xi_0,\ \xi_1$
HEX8	24	7	1 and the six bilinear monomials
TET10	30	4	$1,\ \xi_0,\ \xi_1,\ \xi_2$

Table 2: Rank of the reference tensor. In each case the rank equals the dimension of the span of the element's gradient functions exactly, as Proposition 4 requires.

A linear simplex has rank one because its gradients are constants, and rank one is exactly the classical geometric-factor formulation: one object per element, contracted once. The rank structure is thus not a second mechanism alongside the geometric factor; it is the same structure carried to elements whose gradients are not constant.

Because the rank is known and the entries are rational, the factorisation

$$\overline{W} = C^{\mathsf{T}} X C, \qquad C \in \mathbb{Q}^{r\times Nd}, \qquad X \in \mathbb{Q}^{r\times r}, \tag{10}$$

is computed exactly, with no numerical tolerance and no loss, and verified against the tensor it factors over the rationals.

6 Kernel synthesis

6.1 Staged contraction

Substituting (10) into (5) and naming each intermediate gives four stages:

$$P_{kam} = \sum_{j} C_{a,(j,m)}\, h_{k,j} \qquad \text{compress the increment} \tag{11}$$

$$Y_{ian} = \sum_{k,m} \overline{S}^{0}_{ikmn}\, P_{kam} \qquad \text{apply the tangent} \tag{12}$$

$$Q_{ibn} = \sum_{a} X_{ab}\, Y_{ian} \qquad \text{apply the middle factor} \tag{13}$$

$$\mathrm{out}_{ip} = \sum_{b,n} C_{b,(p,n)}\, Q_{ibn} \qquad \text{expand back to nodes} \tag{14}$$

Every sum skips its zero coefficients, so a vanishing entry of C or X never enters an expression rather than being cancelled out of one afterwards.

The staging is essential rather than incidental. Expanding (11)–(14) into one expression per output and allowing common-subexpression elimination to rediscover the structure returns the dense operation count exactly; the saving resides in keeping the intermediates named.

6.2 Direct contraction, and the choice between the two

Staging is not always the cheaper arrangement. The factorisation must compress the increment and expand the result, and where the rank is a large fraction of the order those two stages cost more than the factorisation saves. The alternative contracts $\overline{W}$ against the increment directly,

$$G_{kmpn} = \sum_{j} \overline{W}_{jmpn}\, h_{k,j} \tag{15}$$

$$\mathrm{out}_{ip} = \sum_{k,m,n} \overline{S}^{0}_{ikmn}\, G_{kmpn}, \tag{16}$$

so that $\overline{S}^{0}$ appears once, in a single 27-term contraction per output in three dimensions, with nothing to compress or expand.

Which of the two is cheaper is a property of the element, so the generator constructs both symbolically and emits the one with the lower operation count (Table 3).

element	staged	direct	r of Nd	selected
TET4	207	342	1 of 12	staged
TET10	1380	1605	4 of 30	staged
HEX8	2610	1731	7 of 24	direct

Table 3: Floating-point operations for the two contraction orderings, counted after common-subexpression elimination. The factorisation ceases to pay once the rank reaches a sufficient fraction of the order.

On HEX8 the choice reduces the emitted application from 2732 to 1801 operations, against 1558 for a hand-written kernel for the same contraction, and raises its measured throughput from 100 to 132 MDOF/s under the conditions of Section 8.6.

The comparison must be made *after* elimination. Measured on the unsimplified expressions the direct ordering appears inferior on every element, because the elimination is precisely what removes its redundancy; a selection rule based on the pre-elimination count would reject it universally.

6.3 Constant folding versus runtime tabulation

The entries of $\overline{W}$, C and X are known at generation time and are therefore emitted as literals. The apparently attractive alternative — storing the distinct values in a runtime table and indexing it — compresses the memory and destroys the arithmetic: the constants cease to be constants, the compiler can no longer fold them, and every multiplication by a zero entry survives into the emitted code.

The effect is large. A tabulated implementation of the TET4 kernel stores a table containing exactly the values 1, -1 and 0, and its kernels then multiply by the zero entry twelve times each; the nominal "97.9% memory saving" buys 141 stored numbers that were 0 and ± 1 at a cost of several hundred floating-point operations.

Table 4 locates the benefit of staging. The saving tracks the rank against the order: TET10 at 4/30 gains 2.9× and TET4 at 1/12 gains 2.4×, whereas QUAD4 at 3/8 breaks even, having insufficient structure to improve on common-subexpression elimination of the dense form. Against the tabulated form the improvement is 7.1× on TET4 and 6.1× on TET10.

6.4 Vectorisation and register pressure

The generated kernels process a block of elements at a time with the lane loop innermost. The staged form suits that layout, for three reasons that are worth separating.

element	r/Nd	dense, folded	staged, folded	runtime table
TRI3	1/6	96	**52**	244
TET4	1/12	486	**207**	1467
QUAD4	3/8	238	238	431
HEX8	7/24	4059	**2610**	6330
TET10	4/30	4018	**1380**	8349

Table 4: Floating-point operations in the element application, counted after common-subexpression elimination. These are symbolic operation counts, not measured run times.

The reference tensor is lane-invariant. C and X are compile-time rationals, identical for every element and hence for every lane. Folded, they appear as immediates: no load, no broadcast, and a multiplication by ± 1 or by a power of two is eliminated entirely. Behind a runtime table the same values become a memory reference whose address is lane-independent — a broadcast load rather than an immediate — and, more damagingly, an opaque value that no longer permits folding. This is the arithmetic component of the cost in Table 4.

The tangent is a per-element stream. $\overline{S}^0$ is 45 numbers per element in three dimensions, held one component-array per index in a structure-of-arrays layout. Each component is then a unit-stride read across the lane loop and vectorises without gather.

The intermediates are lane-width vectors. P, Y and Q are per-element scalars, so in a blocked kernel each occupies one vector register. Their counts (Table 5) are the binding constraint: on HEX8, $63+63+63$ live intermediates at a lane width of eight will not remain in registers and the kernel spills. All three stage sizes are $d^2 r$, so rank controls this as well.

element	r	$\lvert P\rvert$	$\lvert Y\rvert$	$\lvert Q\rvert$	outputs
TRI3	1	4	4	4	6
TET4	1	9	9	9	12
QUAD4	3	12	12	12	8
HEX8	7	63	63	63	24
TET10	4	36	36	36	30

Table 5: Live intermediates per element in each stage. In a blocked kernel each is one vector register, so these are the register-pressure figures.

The two difficulties therefore arrive together and are the same difficulty. On the elements where the projection is exact the register pressure is lowest; on HEX8, where the projection is an approximation, it is highest; and HEX8 is also the element on which the staged form loses to the direct contraction, which requires no intermediates at all. A rank-truncated factorisation, keeping $r' < r$ modes, remains available for elements where staging still wins, at the price of a second and larger approximation.

The final stage writes one value per node per component, exactly as a conventional element kernel does, and reuses the same scatter: atomic accumulation on an unordered traversal, plain accumulation into thread-private scratch on a colour- or block-ordered one.

6.5 Storage format and precision

The stored tensor is the only per-element data the application reads. For a symmetric material in three dimensions it is 45 numbers per element; for the mixed Mooney–Rivlin/Kelvin–Voigt

material of Section 8, which contributes one symmetric and one unsymmetric part, it is 45+81 = 126. Table 6 gives the resulting footprints.

components per element	`fp64`	`fp32`	`fp16`
45 (symmetric)	360	180	90
126 (symmetric + unsymmetric)	1008	504	260

Table 6: Bytes of stored tangent per element. The `fp16` row includes a per-element `fp32` scaling factor, applied to the outputs rather than to the components, which is why it exceeds half the `fp32` figure.

7 Implementation

The scheme is implemented in the code generator of SFEM, an open-source finite-element library. From a symbolic statement of the material the generator derives the flux, differentiates it to obtain S, performs the pullback (2), decides symmetry, integrates $\overline{W}$ exactly, computes the factorisation (10), selects the contraction ordering by operation count, and emits two kernels per element type: an *assembly* kernel that consumes geometry and state and produces the store, and an *application* kernel that consumes the store and the increment. A third, reduced-precision application kernel reads a compressed store with a per-element scaling factor.

The kernels are reachable from the library's operator interface through three methods with defaults that decline, so that existing operators and callers are unaffected: a query reporting whether the approximate path is available for the current mesh and element type, an explicit update that assembles the store from the current state, and an application that takes the increment alone. Making the update explicit is a deliberate choice: the stored tangent remains valid until the next update, which is exactly the reuse the method exists to exploit, and the application takes no state argument at all, so it cannot silently consume a stale one.

8 Numerical experiments

8.1 Experimental protocol

Throughput measurements in the literature are easy to render incomparable, so we state the protocol in full.

Hardware and software. Two machines are used. **M1**: an Apple M1 Max laptop with 8 performance and 2 efficiency cores, Homebrew Clang 20.1.8, OpenMP enabled. **Grace**: one node of a GH200 system (CSCS Alps) with 72 Arm Neoverse-V2 cores. All operators within a comparison are built in the same configuration and run in the same process; figures from different machines or different build configurations are never combined in one table.

Timed region. The timed region contains the operator application and nothing else. Initialisation of the output arrays, allocation, and any correctness checking are performed outside it, and each sample averages many repetitions so that no measurement is charged the cost of OpenMP team creation or a cold cache. This matters more than it might appear: a serial output initialisation placed inside the timed region distorts the single-thread figure negligibly (2.52 against 2.68 MDOF/s) and the eight-thread figure by half (13.04 against 18.75), reporting a parallel speed-up of 5.4× where the true figure is 7.0×. The distortion grows with the thread count, which is the worst possible shape for a scaling study.

Problem size. Throughput is reported only at problem sizes that saturate the machine. The comparison against library operators (Section 8.6) uses 10.4×10^6 degrees of freedom; the scaling study (Section 8.7) uses 206 763. Every table states its size, thread count and machine.

Mesh resolution. Accuracy measurements are taken at non-power-of-two resolutions. On a uniform mesh whose spacing is a power of two the entries of an `fp32` store are exactly representable and the difference between the approximate and exact applications collapses to 10^{-14}, which suggests a conclusion far stronger than the truth (Table 8).

Increment. Convergence rates of the projection error are measured against a white-noise increment, for reasons given in Section 8.4.

Comparison frame. Two frames appear in the literature and they are not interchangeable: calling a kernel directly with the geometry precomputed, and calling it through the library's operator interface as a solve does. At 170 373 degrees of freedom the same kernel measures approximately 5 MDOF/s in the first and 1.46 in the second. Only the second is comparable with library operators, and all figures below are taken in it.

8.2 Verification

Each stage of the construction can be wrong in a way the next cannot detect, so it is verified at four independent points.

1. **The symbolic integration**, against four-point Gauss–Legendre quadrature on QUAD4 and HEX8, entry by entry: agreement to 10^{-16}.
2. **The factorisation**, against the tensor it factors: $C^{\mathsf{T}} X C = \overline{W}$ exactly, over the rationals.
3. **The action**, against (1) integrated independently on a concrete tetrahedron, for linear elasticity and for neo-Hookean Ogden material.
4. **The generated operator**, inside the library, driven as a solver drives it.

Only the third detects an incorrect index convention, and it did: the symmetry (7) was initially implemented as (i, k) against (m, n), which the first two checks and any comparison of the staged form against the dense form all pass, because they share the error. This is the practical reason for stating Proposition 3 with its index pairing made explicit.

8.3 Accuracy of the approximation

Proposition 1 settles the affine simplex case; on quadratic tetrahedra and trilinear hexahedra the error must be measured. We do so by warping the reference configuration by an increasing amount w, verifying $\det F > 0$ at each step, and comparing the approximate neo-Hookean application against the exact one (Table 7).

TET10 is two orders of magnitude worse than HEX8 at comparable warp. Both are quadratic in the geometry, but TET10 carries a quadratic *displacement* whose gradient varies linearly over the cell, and it is exactly that variation which projection onto the constants discards.

8.4 Convergence under refinement

Proposition 2 predicts first-order convergence of the perturbation. Measuring it requires care in the choice of increment. Against a *smooth* increment the TET10 relative error remains at 1.0×10^{-2} across an eightfold refinement and appears not to converge at all. The explanation

warp w	0	0.4	1.6
TET4	1.0e−15	1.0e−15	1.0e−15
TET10	2.4e−14	1.0e−02	4.3e−02
HEX8	2.7e−15	—	3.7e−04

Table 7: Relative error of the approximate application against the exact one, neo-Hookean Ogden material, resolution $n = 16$ ($\approx 2.0 \times 10^5$ dof), M1, one thread. The $w = 0$ column is the control: it recovers Proposition 1 and confirms that the sweep measures the geometry rather than the harness. TET4 is flat across the sweep, as it must be.

is in the metric rather than the operator: for a smooth field the assembled $(\mathcal{H}h)_p$ is a discrete second derivative, so contributions from neighbouring elements cancel and the denominator of the relative error collapses by an additional order, while the per-element projection errors carry independent signs and do not cancel. Against a white-noise increment the same kernel converges at $O(h^{1.07})$, in agreement with Proposition 2.

At $w = 0.4$ the measured sequences are: TET10 smooth $1.07 \times 10^{-2} \to 9.93 \times 10^{-3}$ (flat) against random $2.73 \times 10^{-3} \to 2.95 \times 10^{-4}$; HEX8 smooth $1.16 \times 10^{-3} \to 2.08 \times 10^{-5}$.

8.5 Effect of store precision

Table 8 isolates the store precision from the projection error by taking the measurement on configurations where Proposition 1 applies or the warp is zero, so that the entire discrepancy is representation.

element	$n = 8$	$n = 12$	$n = 16$
TET4	6.1e−16	1.2e−07	1.8e−15
TET10	1.4e−14	1.0e−05	4.8e−14
HEX8	—	6.8e−07	—

Table 8: Relative difference between the approximate application with an `fp32` store and the exact application, at three mesh resolutions. The power-of-two columns are artefacts of exact representability, not measurements of the method; the $n = 12$ column is the honest figure.

The approximate application therefore reproduces the exact one to the precision of its store, and that precision is element-dependent: the quadratic basis of TET10 gives its tangent a wider dynamic range and costs roughly two decimal digits more than TET4 for the same `fp32` store. This is an input to the choice of store precision, not a tolerance to be widened.

8.6 Throughput against library operators

Table 9 compares the generated kernels against the corresponding operators of the library, including a hand-optimised partial-assembly implementation of the same neo-Hookean material which is available for TET10 and HEX8 but not for TET4.

The assembly phases, timed separately at the same problem size, are 0.392 s for the generated store against 0.653 s for the hand-optimised implementation on TET10, and 0.414 s against 0.427 s on HEX8. On TET10 the method is therefore ahead on both phases. On HEX8 the assemblies are level, so the entire remaining difference lies in the contraction.

Break-even. The number of applications per assembly at which the method pays for itself requires a reference that performs no setup of its own, which excludes the hand-optimised operator; the exact matrix-free application is the appropriate denominator. Against it, TET10 breaks

element	hand-optimised PA	exact matrix-free	this work	ratio
TET10	137.4	27.2	**219.6**	1.60× faster
HEX8	$122.5 \pm 7\%$	47.5	**132**	parity
TET4	—	47.9	**116.9**	—

Table 9: Application throughput in MDOF/s. Neo-Hookean Ogden material, 1.04×10^7 degrees of freedom, M1, eight threads, identical build for all three columns. The HEX8 entries are five-run means; the run-to-run spread of the hand-optimised operator on that element is ±7%, so the comparison there is a statistical tie. Before the contraction-ordering selection of Section 6.2 the HEX8 figure for this work was 100.2.

even at 1.03 applications per assembled tangent and TET4 at 1.16. Since a Krylov method performs tens to hundreds of applications per Newton step, the assembly cost is amortised after the first iteration.

8.7 Parallel scalability

machine	variant	1	8	32	64	72
M1	exact matrix-free	2.68	18.75	—	—	—
M1	stored, `fp16`	7.61	55.15	—	—	—
Grace	exact matrix-free	1.33	10.68	41.65	82.13	92.28
Grace	stored, `fp16`	2.85	25.30	97.67	191.62	209.60

Table 10: Throughput in MDOF/s against thread count, Mooney–Rivlin material with Kelvin–Voigt viscosity, 206 763 degrees of freedom. Grace attains 69.4× on 72 cores for the exact application and 73.5× for the stored one.

On Grace the scaling is essentially linear to the full node, with parallel efficiency of 96% for the exact application and above unity for the stored one, the latter reflecting improved cache residency as the per-thread working set shrinks. On M1 every variant peaks at eight threads and falls by approximately a fifth at ten. This is a property of the machine, not of the scatter: the M1 Max has eight performance and two efficiency cores, a static loop schedule assigns them equal chunks, and the loop waits for the slower ones. An earlier reading of the same data attributed the plateau to contention in the atomic scatter; the Grace measurements refute that reading, and we record it as a caution against inferring an algorithmic limit from a single heterogeneous machine.

Reduced precision does not pay on either machine. At one thread the `fp16` store is slower than `fp32`, and at 72 threads it is within two per cent of it while costing four decimal digits of accuracy. The kernel is not bandwidth-bound at the point where halving the store would help, and the conversion cost is not recovered. `fp32` is the operating point we recommend.

9 Discussion and limitations

Where the method is expected to pay. The method converts state-dependent work into a one-off assembly plus a state-independent application. It is therefore favourable exactly when the ratio of applications to assemblies is high — Krylov iterations within a Newton step, repeated smoothing at a fixed state within a multigrid cycle — and unfavourable when the state changes every application. The measured break-even below 1.2 applications means the threshold is essentially always met in a Newton–Krylov context.

Where the approximation is inadmissible. The scheme perturbs the operator by $O(h)$. Used as the operator of record it changes the discrete problem being solved. Its proper roles are as the Krylov operator inside an inexact-Newton iteration, where the residual is still evaluated exactly, and as a preconditioner. Quantifying the effect on Newton convergence for a specific forcing-term strategy is the most immediate open question and is not addressed here.

Element coverage. The trilinear hexahedron remains behind the hand-optimised kernel in the contraction, though not in the assembly; the gap is 1801 against 1558 operations and closing it is a matter of the contraction schedule rather than of the method. The two-dimensional kernels are generated and verified symbolically but have not been executed, pending support for the required affine geometry cache on triangular and quadrilateral meshes in the underlying mesh library.

Higher order. Proposition 4 gives the rank for any element, and it grows with the polynomial degree; at sufficiently high order the rank approaches the order, the factorisation ceases to pay (Table 3 already shows this trend at HEX8), and sum factorisation becomes the appropriate technique instead. Determining the crossover degree is future work.

10 Conclusions

Projecting the pulled-back material tangent onto element-wise constants removes the quadrature loop from the matrix-free application of a nonlinear elasticity operator, leaving an application that reads only a small stored tensor and the increment. The reference tensor that remains is a Gram matrix whose rank is the dimension of the span of the element's gradient functions, and can be factored exactly in rational arithmetic and folded into the emitted code as literal constants. The projection is exact on affine simplices for arbitrary hyperelastic materials and first-order accurate otherwise, at a measured rate of $O(h^{1.07})$.

At 1.04×10^7 degrees of freedom the generated kernels are $1.60\times$ faster than a hand-optimised partial-assembly implementation of the same material on quadratic tetrahedra, with a $1.67\times$ faster assembly, and at parity on trilinear hexahedra; parallel efficiency on 72 Arm cores is 96%. The symmetry that halves the storage must be decided per material rather than assumed, since it holds for energy-based formulations and fails for residual-based ones.

Three directions follow. The first is the inexact-Newton analysis: with an $O(h)$ Jacobian perturbation, what forcing-term strategy retains superlinear convergence, and how does the resulting iteration count trade against the per-application saving? The second is the use of the stored tensor as a multigrid smoother operator, where the ratio of applications to assemblies is higher still. The third is rank truncation: Proposition 4 makes the modes explicit, and discarding the smallest is a controlled second approximation whose error can be estimated a priori.

Acknowledgements

To be completed.

References

[1] A. Abdelfattah et al. A survey of numerical linear algebra methods utilizing mixed-precision arithmetic. *Int. J. High Perform. Comput. Appl.*, 35(4):344–369, 2021.

[2] R. Anderson et al. MFEM: A modular finite element methods library. *Comput. Math. Appl.*, 81:42–74, 2021.

[3] M. S. Alnæs, A. Logg, K. B. Ølgaard, M. E. Rognes, G. N. Wells. Unified Form Language: A domain-specific language for weak formulations of partial differential equations. *ACM Trans. Math. Softw.*, 40(2):9, 2014.

[4] J. Bonet, R. D. Wood. *Nonlinear Continuum Mechanics for Finite Element Analysis*. Cambridge University Press, 2nd edition, 2008.

[5] J. H. Bramble, S. R. Hilbert. Estimation of linear functionals on Sobolev spaces with application to Fourier transforms and spline interpolation. *SIAM J. Numer. Anal.*, 7(1):112–124, 1970.

[6] J. Brown. Efficient nonlinear solvers for nodal high-order finite elements in 3D. *J. Sci. Comput.*, 45(1–3):48–63, 2010.

[7] J. Brown et al. libCEED: Fast algebra for high-order element-based discretizations. *J. Open Source Softw.*, 6(63):2945, 2021.

[8] P. G. Ciarlet. *The Finite Element Method for Elliptic Problems*. North-Holland, 1978.

[9] R. S. Dembo, S. C. Eisenstat, T. Steihaug. Inexact Newton methods. *SIAM J. Numer. Anal.*, 19(2):400–408, 1982.

[10] M. O. Deville, P. F. Fischer, E. H. Mund. *High-Order Methods for Incompressible Fluid Flow*. Cambridge University Press, 2002.

[11] S. C. Eisenstat, H. F. Walker. Choosing the forcing terms in an inexact Newton method. *SIAM J. Sci. Comput.*, 17(1):16–32, 1996.

[12] N. J. Higham, T. Mary. Mixed precision algorithms in numerical linear algebra. *Acta Numerica*, 31:347–414, 2022.

[13] R. C. Kirby, A. Logg. A compiler for variational forms. *ACM Trans. Math. Softw.*, 32(3):417–444, 2006.

[14] D. A. Knoll, D. E. Keyes. Jacobian-free Newton–Krylov methods: a survey of approaches and applications. *J. Comput. Phys.*, 193(2):357–397, 2004.

[15] T. G. Kolda, B. W. Bader. Tensor decompositions and applications. *SIAM Rev.*, 51(3):455–500, 2009.

[16] T. Kolev et al. Efficient exascale discretizations: High-order finite element methods. *Int. J. High Perform. Comput. Appl.*, 35(6):527–552, 2021.

[17] M. Kronbichler, K. Kormann. A generic interface for parallel cell-based finite element operator application. *Comput. Fluids*, 63:135–147, 2012.

[18] M. Kronbichler, K. Kormann. Fast matrix-free evaluation of discontinuous Galerkin finite element operators. *ACM Trans. Math. Softw.*, 45(3):29, 2019.

[19] J. M. Melenk, K. Gerdes, C. Schwab. Fully discrete *hp*-finite elements: fast quadrature. *Comput. Methods Appl. Mech. Engrg.*, 190(32–33):4339–4364, 2001.

[20] R. W. Ogden. Large deformation isotropic elasticity — on the correlation of theory and experiment for incompressible rubberlike solids. *Proc. R. Soc. Lond. A*, 326:565–584, 1972.

[21] S. A. Orszag. Spectral methods for problems in complex geometries. *J. Comput. Phys.*, 37(1):70–92, 1980.

[22] F. Rathgeber et al. Firedrake: Automating the finite element method by composing abstractions. *ACM Trans. Math. Softw.*, 43(3):24, 2016.